

CS 170 Programming Quiz 5



For **PQ05 - Methods and Decisions**, you'll write two methods: one that uses `String` methods, and one that uses selection. This exam covers the last part of Chapter 4 and Chapter 5 in your textbook. For this exam you'll need to know:

- The correct syntax for defining a method (writing the skeleton or stub)
- How to declare parameters correctly (type and order)
- How to call the methods in the `String` class (`substring`, `indexOf`, `charAt`, `length`)
- How to correctly return values from a method
- How to write single and multi-way decision statements

The method will be tested for syntax correctness (name, `public`, return type and parameters) and for correctness. Work step-by-step so that you get partial credit. You can find more problems to practice with on Codingbat in the `String 1` and `Logic2` sections. You'll find the links on Canvas.

These are a sample of the kind of problems you'll find, not necessarily the problems themselves.

1 THE OUTSIDEIN3 PROBLEM

Write the method named `outsideIn3`. The method has one input: a `String str` that is at least 3 characters long and of odd length. The output of the method is a new `String` where the middle character has been removed and placed both at the beginning and end. Here is some sample input and output:

- `outsideIn3("Cartons")` → `"tCaronst"` // *t removed and placed at both ends*
- `outsideIn3("Hip")` → `"iHpi"` // *i removed placed at both ends.*
- `outsideIn3("Barbara Ann")` → `"rBarbaa Annr"` // *2nd r removed*

2 THE INSIDEOUT3 PROBLEM

Write the method named `insideOut3`. The method has one input: a `String str` that is at least 3 characters long and of odd length. The output of the method is a new `String` where the middle three characters have been removed. Here is some sample input and output:

- `insideOut3("Cartons")` → `"Cans"` *// rto removed*
- `insideOut3("Hip")` → `" "` *//all three characters removed.*
- `insideOut3("Barbara Ann")` → `"Barb Ann"` *//ara removed*

3 THE OUTSIDEIN2 PROBLEM

Write the method named `outsideIn2`. The method has one input: a `String str` that is at least 2 characters long and of even length. The output of the method is a new `String` where the middle two characters have been removed and placed at the beginning of the result. Here is some sample input and output:

- `outsideIn2("Carton")` → `"rtCaon"` *// rt moves to front*
- `outsideIn2("Hi")` → `"Hi"` *// Hi moved to from so output unchanged*
- `outsideIn2("Barbara Anne")` → `"raBarba Anne"` *// ra moved to front*

4 THE LEFTN PROBLEM

Write the method `leftN`. The method has two inputs: a `String str` and an `int n`. The output of the method is a new `String` that contains a "rotated left n" version of `str`, where the first `n` chars are moved to the end of the `String`. The original `String` length will be at least `n`. Here is some sample input and output:

- `leftN("Hello", 2)` → `"lloHe"`
- `leftN("Chocolate", 3)` → `"colateCho"`
- `leftN("Chocolate", 1)` → `"hocolateC"`

5 THE MIDDLETHREE PROBLEM

Write the method `middleThree`. The method has one input: an odd-length `String str` of at least length 3. The output of the method is a new `String` that contains the middle three characters, so calling the method with the `String "Candy"` returns `"and"`. Here is some sample input and output:

- `middleThree("Candy")` → `"and"`
- `middleThree("and")` → `"and"`
- `middleThree("solving")` → `"lvi"`

6 THE NTWICE PROBLEM

Write the method `nTwice`. The method has two inputs: a `String str` and an `int n`. The output of the method is a new `String` made of the first and last `n` chars from `str`, where the last `n` chars are moved to the front of the `String`. The original `String` length will be at least length `n`. Here is some sample input and output:

- `rightN("Hello", 2) → "loHel"`
- `rightN("Chocolate", 3) → "ateChocol "`
- `rightN("Chocolate", 1) → "eChocolat"`

7 THE RIGHTN PROBLEM

Write the method `rightN`. The method has two inputs: a `String str` and an `int n`. The output of the method is a new `String` that contains a "rotated right `n`" version of `str`. You can assume that the length of `str` will be at least `n`. Here is some sample input and output:

- `nTwice("Hello", 2) → "Helo"`
- `nTwice("Chocolate", 3) → "Choate"`
- `nTwice("Chocolate", 1) → "Ce"`

8 THE INSIDEOUT2 PROBLEM

Write the method named `insideOut2`. The method has one input: a `String str` that is at least 2 characters long and of even length. The output of the method is a new `String` where the middle two characters have been removed and placed at the end of the result. Here is some sample input and output:

- `insideOut2 ("Carton")→ "Caonrt" // rt moved to end`
- `insideOut2 ("Hi")→ "Hi" // Hi moved to end, so no change.`
- `insideOut2 ("Barbara Anne")→ "Barba Annera" //ra moves end.`

9 A LOTTERY PROBLEM

Write the method named `lottery`. You have a lottery ticket, with `ints a, b, and c` on it. This makes three pairs, which we'll call `ab, bc, and ac`. Consider the sum of the numbers in each pair. If any pair sums to exactly 10, the result is 10. Otherwise if the `ab` sum is exactly 10 more than either `bc` or `ac` sums, the result is 5. Otherwise the result is 0. Return the sum. Here are some samples.

- input of 9, 1, 0 → 10
- input of 9, 2, 0 → 0
- input of 6, 1, 4 → 10

10 THE LIMITING PROBLEM

Write the method named `limitSum`. Given 2 non-negative `ints`, `a` and `b`, return their sum, so long as the sum has the same number of digits as `a`. If the sum has more digits than `a`, just return `a` without `b`.

- input of 2, 3 → 5
- input of 8, 3 → 8
- input of 8, 1 → 9

11A LOTTERY PROBLEM

Write a method named `lottery`. You have a lottery ticket, with `ints` `a`, `b`, and `c` on it. If the numbers are all different from each other, the result is 0. If all of the numbers are the same, the result is 20. If two of the numbers are the same, the result is 10. Return the result.

- input of 1, 2, 3 → 0
- input of 2, 2, 2 → 20
- input of 1, 1, 2 → 10

12A LOTTERY PROBLEM

Write a method named `lottery`. You have a lottery ticket showing `ints` `a`, `b`, and `c`, each of which is 0, 1, or 2. If they are all the value 2, the result is 10. Otherwise if they are all the same, the result is 5. Otherwise so long as both `b` and `c` are different from `a`, the result is 1. Otherwise the result is 0. Return the result.

- input of 2, 2, 2 → 10
- input of 2, 2, 1 → 0
- input of 0, 0, 0 → 5

13 THE MAX FIVE PROBLEM

Write a method named `maxFive`. Given two `int` values, return whichever value is larger. However, if the two values have the same remainder when divided by 5, then the return the smaller value. However, in all cases, if the two values are the same, return 0.

- input of 2, 3 → 3
- input of 6, 2 → 6
- input of 3, 2 → 3

14 THE DICE PROBLEM

Write a method named `rollEm`. Return the sum of two 6-sided dice rolls, each in the range 1..6. However, if `noDoubles` (the third argument) is `true`, and, if the two dice show the same value, increment one die to the next value, wrapping around to 1 if its value was 6. Here are some examples.

- input of 2, 3, `true` → 5
- input of 3, 3, `true` → 7
- input of 3, 3, `false` → 6

15A FIZZY PROBLEM

Write a method named `fizzBuzz`. Given an `int n`, return the number followed by "!". So the `int 6` returns "6!". Except if the number is divisible by 3 use "Fizz" instead of the number, and if the number is divisible by 5 use "Buzz", and if divisible by both 3 and 5, use "FizzBuzz". Do not put the quotes around your result.

- input of 1 → "1!"
- input of 2 → "2!"
- input of 3 → "Fizz!"

16 ANOTHER FIZZY PROBLEM

Write a method named `fizzBuzz`. Given an input string `str`, if the string starts with "f" (either upper or lowercase), return "Fizz". If the string ends with "b" (either upper or lowercase), return "Buzz". If both the "f" and "b" conditions are `true`, return "FizzBuzz". In all other cases, return the string unchanged. Do not put the quotes around your result.

- input of "fig" → "Fizz"
- input of "dib" → "Buzz"
- input of "fib" → "FizzBuzz"

17 THE PARTY PROBLEM

Write a method named `partyOn`. We are having a party with amounts of tea and crumpets. Return the outcome of the party as "sad", "pleasing" or "brilliant". A party is pleasing if both tea and crumpets are at least 5. However, if either tea or crumpets is at least double the amount of the other one, the party is brilliant. However, in all cases, if either tea or crumpets is less than 5, the party is always sad. Do not put the quotes around your result.

- input of 6, 8 → "pleasing"
- input of 3, 8 → "sad"
- input of 20, 6 → "brilliant"

18 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the two `int` arguments, `a` and `b`. However, "teen" values in the range 13..19 inclusive, are extra lucky. So if either value is a teen, just return 19.

- input of 3, 4 → 7
- input of 10, 13 → 19
- input of 13, 2 → 19

19 THE VACATION PROBLEM

Write a method named `vacation`, which takes two arguments: a day of the week encoded as 0=Sun, 1=Mon, 2=Tue, ...6=Sat, and a boolean indicating if you are on vacation. Return a String of the form "7:00" indicating when your alarm clock should ring.

On weekdays, the alarm should be "7:00" and on the weekend it should be "10:00". Unless you are on vacation; then on weekdays it should be "10:00" and weekends it should be "off". Don't put quotes around your answer.

- input of 1, false → "7:00"
- input of 5, false → "7:00"
- input of 0, false → "10:00"

20 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the two `int` arguments, `a` and `b`. However, sums in the range 10..19 inclusive, are forbidden, so in that case just return 20.

- input of 3, 4 → 7
- input of 9, 4 → 20
- input of 10, 11 → 21

21 THE TICKET PROBLEM

You are driving a little too fast, and a police officer stops you. Write a method named `ticket` that determines what happens: you may get no ticket (0), a small ticket (25) or a big ticket (275). If your speed is 60 or less, there is no ticket. If your speed is between 61 and 80 inclusive, the result is a small ticket. If your speed is 81 or more, the result is a large ticket. Unless it is your birthday; on that day, your speed can be 5 higher in all cases.

- input of 60, birthday=false → 0
- input of 65, birthday=false → 25
- input of 65, birthday=true → 0

22 THE STYLIN' PROBLEM

Write a method named `reservation`. You and your date other are trying to get a table at a hot new restaurant. The first input ("you"), represents the stylishness of your clothes in the range 0..10; the second ("date") is the stylishness of your date's clothes. The method returns "no", "maybe", or "yes". If either of you is very stylish, 8 or more, then the result is yes. With the exception that if either of you has style of 2 or less, then the result is no. Otherwise the result is maybe.

- input of 5, 10 → "yes"
- input of 5, 2 → "no"
- input of 5, 5 → "maybe"

23 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the three `int` arguments, `a`, `b` and `c`. What's the catch? If one of the values is the same as any other of the three values, it does not count towards the sum.

- input of 1, 2, 3 → 6
- input of 3, 2, 3 → 2
- input of 3, 3, 3 → 0

24 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the three `int` arguments, `a`, `b` and `c`. What's the catch? If one of the values is 13 then it does not count towards the sum and values to its right do not count. So for example, if `b` is 13, then both `b` and `c` do not count.

- input of 1, 2, 3 → 6
- input of 1, 2, 13 → 3
- input of 1, 13, 3 → 1

25 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the three `int` arguments, `a`, `b` and `c`. What's the catch? If any of the values is a teen-in the range 13..19 inclusive- then that value counts as 0, except 15 and 16 do not count as a teens. (You may write a separate helper method to determine that, if you like.)

- input of 1, 2, 3 → 6
- input of 2, 13, 1 → 3
- input of 2, 1, 14 → 3

26 THE SPECIAL SUM PROBLEM

Write a method named `specialSum`. Return the sum of the rounded values of three `int` arguments, `a`, `b` and `c`. We'll round an `int` value up to the next multiple of 10 if its rightmost digit is 5 or more, so 15 rounds up to 20. Alternately, round down to the previous multiple of 10 if its rightmost digit is less than 5, so 12 rounds down to 10. To avoid code repetition, you may write a separate helper method but that is not required.

- input of 16, 17, 18 → 60 *// note, 16-→20 + 17-→20 + 18-→20 -> 60*
- input of 12, 13, 14 → 30 *// note, 12-→10 + 13-→10 + 14-→10 -> 30*
- input of 6, 4, 4 → 10 *// note, 6-→10 + 4-→0 + 4-→0 ->10*

27 THE BLACKJACK PROBLEM

Write a method named `blackJack` that is given 2 `int` values, both of which are greater than 0. Your method should return whichever value is nearest to 21 without going over. Return 0 if they both go over.

- input of 19, 21 → 21
- input of 21, 19 → 21
- input of 19, 22 → 19

28 A FULLFILLING PROBLEM

You are a small candy shop that creates custom candy boxes. Write a method named `fillOrder` that takes three arguments (all `int`): the number of small chocolates (1 gram each) you have in stock, the number of large chocolates (5 grams each) you have in stock, and the goal for customer's order. Return the number of small bars to use in the box, assuming you always use big bars before small bars. Return -1 if it can't be done.

- for input of `small=4, large=1, goal=9` → 4
- for input of `small=4, large=1, goal=10` → -1
- for input of `small=4, large=1, goal=7` → 2

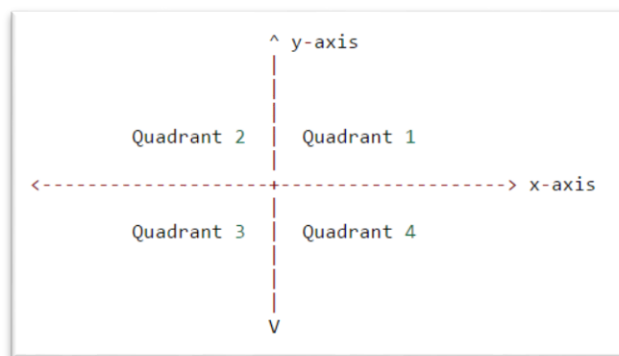
29 THE TRIANGLE TYPE PROBLEM

Can you remember which kind of triangle is which? I can't. Write a method named `tType` to help you out. Given three integers (`a`, `b` and `c`) representing the lengths of the sides of a triangle, return what type of triangle it is. The three types are equilateral, isosceles, and scalene. An equilateral triangle has all three sides the same length, an isosceles triangle has two sides the same length, and a scalene triangle has three sides of different lengths. Here are some examples:

- for input of 5, 7, 7 → isosceles
- for input of 6, 6, 6 → equilateral
- for input of 5, 7, 8 → scalene
- for input of 12, 18, 12 → isosceles

30 THE QUADRANT PROBLEM

Write a method named `quadrant`. Given a pair of real numbers representing an (x, y) point, return the quadrant number for that point. Recall that quadrants are numbered as integers from 1 to 4 with the upper-right quadrant numbered 1 and the subsequent quadrants numbered in a counter-clockwise fashion:



Notice that the quadrant is determined by whether the x and y coordinates are positive or negative numbers. If a point falls on the x -axis or the y -axis, then quadrant is 0.

Here are some samples:

- for input of 12.4, 17.8 → 1
- for input of -2.3, 3.5 → 2
- for input of -15.2, -3.1 → 3
- for input of 4.5, -42.0 → 4
- for input of 0.0, 3.14 → 0